

Технологии и методы программирования

Ищенко Алексей Петрович

Технология программирования:

основные этапы развития

Технология программирования – это совокупность методов и средств, используемых в процессе разработки программного обеспечения. Технология программирования представляет собой набор технологических инструкций, включающий:

- указание последовательности выполнения технологических операций;
- перечисление условий, при которых выполняется та или иная операция;
- описания самих операций, в которых для каждой операции определены исходные данные, результаты, а также инструкции, нормативы, стандарты, критерии и методы оценки и т.п.

Структура описания технологической операции

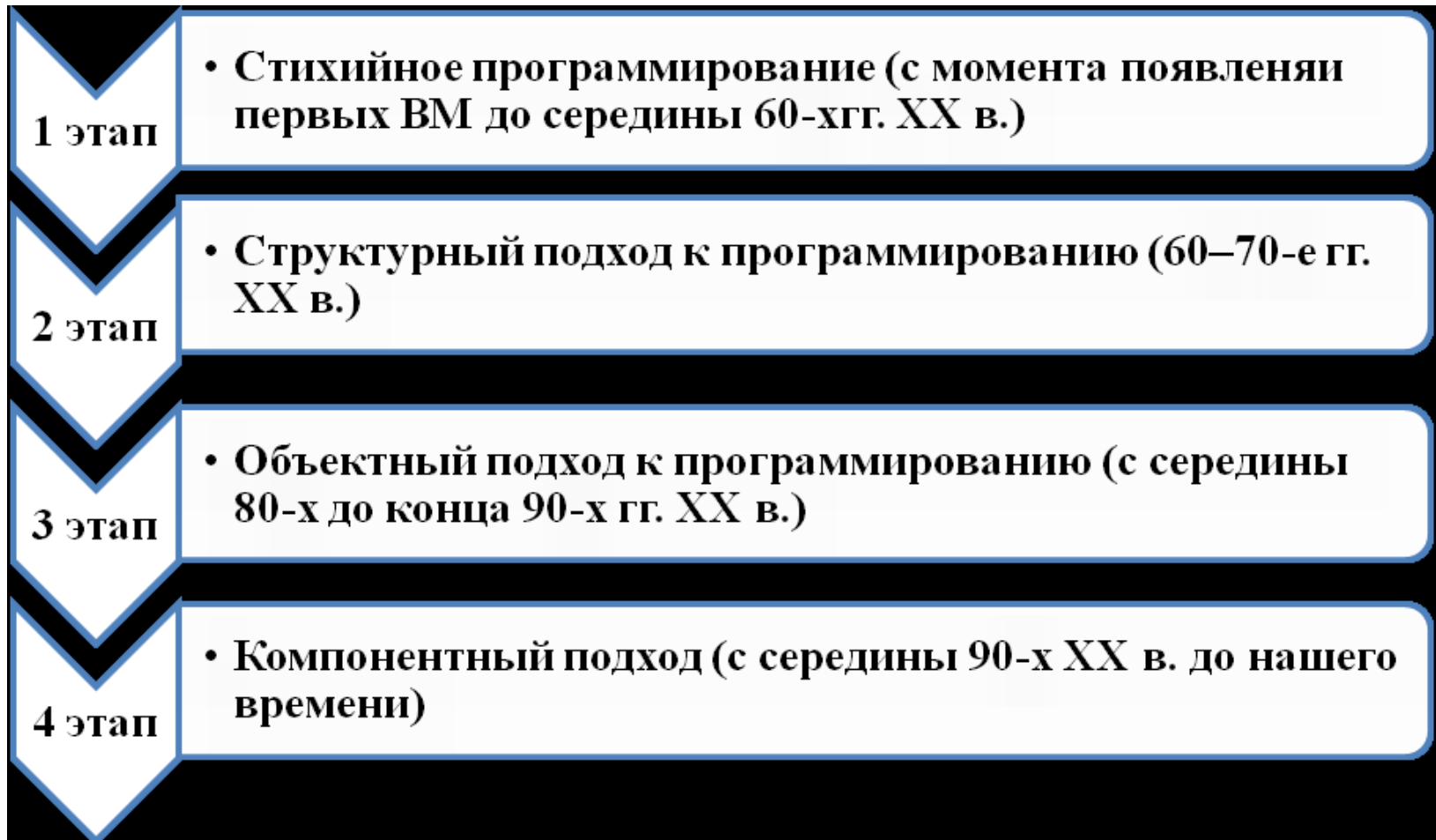


Технология программирования:

основные этапы развития

- Кроме набора операций и их последовательности, технология программирования также определяет способ описания проектируемой системы, используемой на конкретном этапе разработки.
- Выделяют технологии программирования, используемые на конкретных этапах разработки или для решения отдельных задач этих этапов, и технологии, охватывающие несколько этапов или весь процесс разработки. В основе первых, как правило, лежит ограниченно применимый метод, позволяющий решить конкретную задачу, в основе вторых – базовый метод или подход, определяющий совокупность методов, используемых на разных этапах разработки, или методологию.

Этапы развития программирования



Первый этап – «стихийное» программирование

- Этот этап охватывает период от момента появления первых вычислительных машин до середины 60-х гг. XX в. В этот период практически отсутствовали сформулированные технологии и программирование фактически было искусством. Первые программы имели простейшую структуру. Они состояли из собственно программы на машинном языке и обрабатываемых ею данных.

Первый этап – «стихийное» программирование

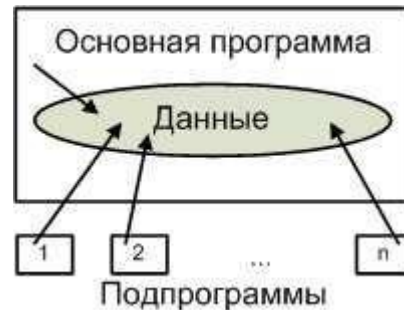


- Сложность программ в машинных кодах ограничивалась способностью программиста одновременно мысленно отслеживать последовательность выполняемых операций и местонахождение данных при программировании.
- Появление ассемблеров позволило вместо двоичных или 16-ричных кодов использовать символические имена данных и мнемоники кодов операций. В результате программы стали более «читаемыми».

Первый этап – «стихийное» программирование

- Создание языков программирования высокого уровня, таких как FORTRAN и ALGOL, существенно упростило программирование вычислений, снизив уровень детализации операций. Это, в свою очередь, позволило увеличить сложность программ.
- Революционным было появление в языках средств, дающих возможность оперировать подпрограммами (идея написания подпрограмм возникла гораздо раньше, но отсутствие средств поддержки в первых языковых средствах значительно снижало эффективность их применения). Подпрограммы можно было сохранять и использовать в других программах. В результате были созданы огромные библиотеки расчетных и служебных подпрограмм, которые по мере надобности вызывали из разрабатываемой программы.

Архитектура программы с глобальной областью данных

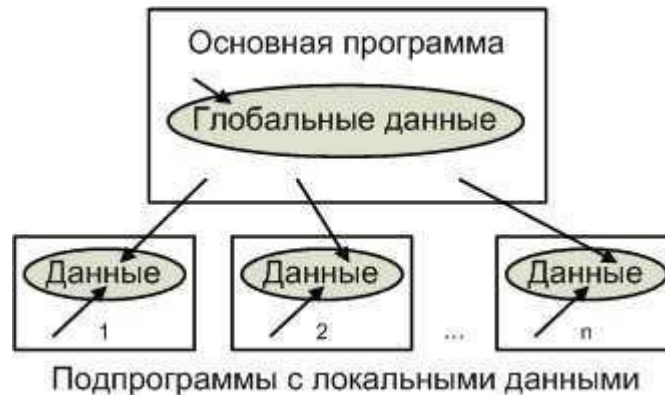


- Типичная программа того времени состояла из основной программы, области *глобальных данных* и набора подпрограмм (в основном библиотечных), выполняющих обработку всех данных или их части

Архитектура программы с глобальной областью данных

- Слабым местом такой архитектуры было то, что при увеличении количества подпрограмм возрастала вероятность искажения части глобальных данных какой-либо подпрограммой. Например, подпрограмма поиска корней уравнения на заданном интервале по методу деления отрезка пополам меняет величину интервала. Если при выходе из подпрограммы не предусмотреть восстановления первоначального интервала, то в глобальной области окажется неверное значение интервала. Чтобы сократить количество таких ошибок, было предложено в подпрограммах размещать *локальные данные*

Архитектура программы, использующей подпрограммы с локальными данными



- Чтобы сократить количество таких ошибок, было предложено в подпрограммах размещать *локальные данные*

Первый этап – «стихийное» программирование

- Сложность разрабатываемого программного обеспечения при использовании подпрограмм с локальными данными по-прежнему ограничивалась возможностью программиста отслеживать процессы обработки данных, но уже на новом уровне. Однако появление средств поддержки подпрограмм позволило осуществлять разработку программного обеспечения несколькими программистами параллельно.
- В начале 60-х гг. XX в. разразился «кризис программирования». Он выражался в том, что фирмы, взявшиеся за разработку сложного программного обеспечения, такого как операционные системы, срывали все сроки завершения 10 проектов. Проект устаревал раньше, чем был готов к внедрению, увеличивалась его стоимость, и в результате многие проекты так никогда и не были завершены.

Первый этап – «стихийное» программирование

- Объективно все это было вызвано несовершенством технологии программирования. Прежде всего стихийно использовалась разработка «снизу вверх» – подход, при котором вначале проектировали и реализовывали сравнительно простые подпрограммы, из которых затем пытались построить сложную программу.
- В отсутствие четких моделей описания подпрограмм и методов их проектирования создание каждой подпрограммы превращалось в непростую задачу, интерфейсы подпрограмм получались сложными, и при сборке программного продукта выявлялось большое количество ошибок согласования.
- Исправление таких ошибок, как правило, требовало серьезного изменения уже разработанных подпрограмм, что еще более усложняло ситуацию, так как при этом в программу часто вносились новые ошибки, которые также необходимо было исправлять. В конечном счете процесс тестирования и отладки программ занимал более 80 % времени разработки, если вообще когда-нибудь заканчивался.

Второй этап – структурный подход к программированию

- Структурный подход к программированию представляет собой совокупность рекомендуемых технологических приемов, охватывающих выполнение всех этапов разработки программного обеспечения.
- В основе структурного подхода лежит **декомпозиция** (разбиение на части) сложных систем с целью последующей реализации в виде отдельных небольших (до 40–50 операторов) подпрограмм.
- С появлением других принципов декомпозиции (объектного, логического и т.д.) данный способ получил название *процедурной* декомпозиции.

Структурный подход к программированию

- В отличие от используемого ранее процедурного подхода к декомпозиции структурный подход требовал представления задачи в виде иерархии подзадач простейшей структуры. Проектирование, таким образом, осуществлялось «сверху вниз» и подразумевало реализацию общей идеи, обеспечивая проработку интерфейсов подпрограмм. Одновременно вводились ограничения на конструкции алгоритмов, рекомендовались формальные модели их описания, а также специальный метод проектирования алгоритмов – метод пошаговой детализации.

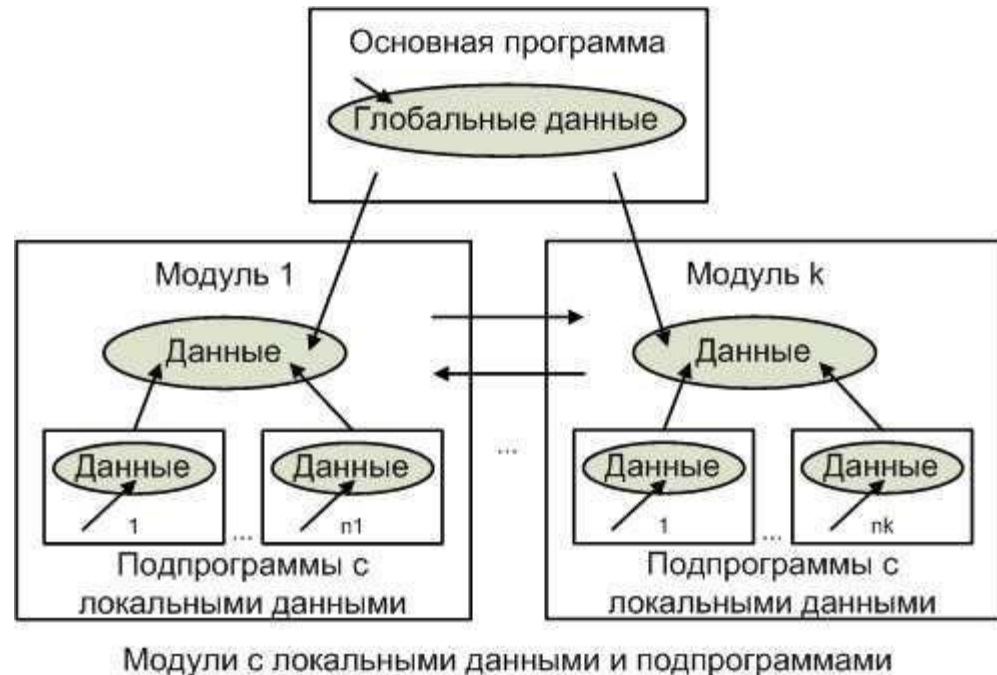
Структурный подход к программированию

- Поддержка принципов структурного программирования была заложена в основу так называемых *процедурных* языков программирования. Как правило, они включали основные «структурные» операторы передачи управления, поддерживали вложение подпрограмм, локализацию и ограничение области «видимости» данных. Среди наиболее известных языков этой группы стоит назвать PL/1, ALGOL-68, Pascal, C.
- Одновременно со структурным программированием появилось огромное количество языков, базирующихся на других концепциях, но большинство из них не выдержало конкуренции. Какие-то языки были просто забыты, идеи других были использованы в следующих версиях развиваемых языков.

Модульное программирование

- Дальнейший рост сложности и размеров разрабатываемого программного обеспечения потребовал развития *структурирования данных*. Как следствие этого в языках появляется возможность определения пользовательских типов данных. Одновременно усилилось стремление разграничить доступ к глобальным данным программы, чтобы уменьшить количество ошибок, возникающих при работе с глобальными данными. В результате появилась и начала развиваться технология модульного программирования.
- *Модульное программирование* предполагает выделение групп подпрограмм, использующих одни и те же глобальные данные в отдельно компилируемые *модули* (библиотеки подпрограмм), например модуль графических ресурсов, модуль подпрограмм вывода на принтер

Архитектура программы, состоящей из модулей



Связи между модулями при использовании данной технологии осуществляются через специальный интерфейс, в то время как доступ к реализации модуля (телам подпрограмм и некоторым «внутренним» переменным) запрещен.

Модульное программирование

- Использование модульного программирования существенно упростило разработку программного обеспечения несколькими программистами. Теперь каждый из них мог разрабатывать свои модули независимо, обеспечивая взаимодействие модулей через специально оговоренные межмодульные интерфейсы. Кроме того, модули в дальнейшем без изменений можно было использовать в других разработках, что повысило производительность труда программистов.
- Практика показала, что структурный подход в сочетании с модульным программированием позволяет получать достаточно надежные программы, размер которых не превышает 100 000 операторов. Узким местом модульного программирования служит то, что ошибка в интерфейсе при вызове подпрограммы выявляется только при выполнении программы (из-за отдельной компиляции модулей обнаружить эти ошибки раньше невозможно).

Модульное программирование

- При увеличении размера программы обычно возрастает сложность межмодульных интерфейсов, и с некоторого момента предусмотреть взаимовлияние отдельных частей программы становится практически невозможно. Для разработки программного обеспечения большого объема было предложено использовать объектный подход.

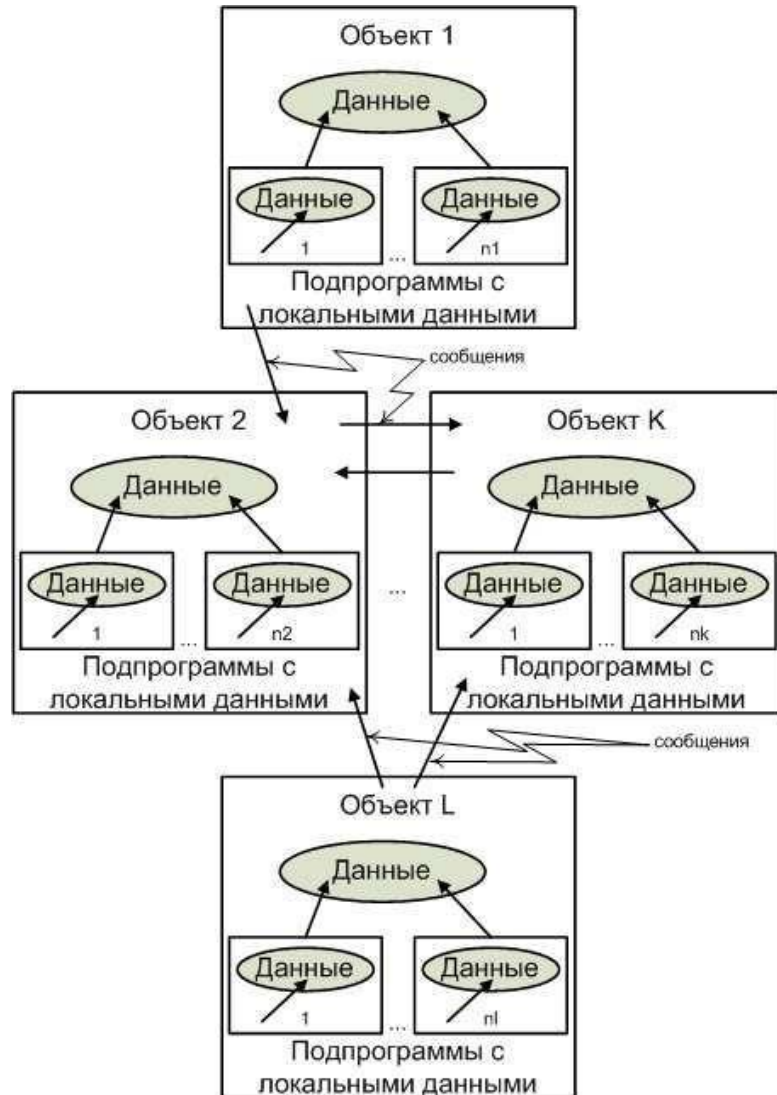
Третий этап – объектный подход к программированию

- Объектно-ориентированное программирование – технология создания сложного программного обеспечения, основанная на представлении программы в виде совокупности *объектов*, каждый из которых является экземпляром определенного типа (*класса*), а классы образуют иерархию с *наследованием* свойств. Взаимодействие программных объектов в такой системе осуществляется путем передачи *сообщений*.

Объектный подход к программированию

- Основным достоинством объектно-ориентированного программирования по сравнению с модульным программированием является «более естественная» декомпозиция программного обеспечения, которая существенно облегчает его разработку. Это приводит к более полной локализации данных и интегрированию их с подпрограммами обработки, что позволяет вести практически независимую разработку отдельных частей (объектов) программы. Кроме этого, объектный подход предлагает новые способы организации программ, основанные на механизмах наследования, полиморфизма, композиции, наполнения. Эти механизмы дают возможность конструировать сложные объекты из сравнительно простых. В результате существенно увеличивается показатель повторного использования кодов и появляется возможность создания библиотек классов для различных применений

Архитектура программ при объектно-ориентированном программировании



Объектный подход к программированию

- Бурное развитие технологий программирования, основанных на объектном подходе, позволило решить многие проблемы. Так были созданы среды, поддерживающие визуальное программирование, например Delphi, C++ Builder, Visual C++ и т.д.
- При использовании визуальной среды у программиста появляется возможность проектировать некоторую часть, например интерфейсы будущего продукта, с применением визуальных средств добавления и настройки специальных библиотечных компонентов. Результат визуального проектирования – заготовка будущей программы, в которую уже внесены соответствующие коды.

Объектный подход к программированию

- Можно дать обобщающее определение: *объект ООП* – это совокупность переменных состояния и связанных с ними методов (операций). Упомянутые методы устанавливают, как объект взаимодействует с окружающим миром.
- Под *методами объекта* понимают процедуры и функции, объявление которых включено в описание объекта и которые выполняют действия. Возможность управлять состояниями объекта посредством вызова методов в итоге и определяет поведение объекта. Эту совокупность методов часто называют интерфейсом объекта.
- *Инкапсуляция* – это механизм, который объединяет данные и методы, манипулирующие этими данными, и защищает и то и другое от внешнего вмешательства или неправильного использования. Когда методы и данные объединяются таким способом, создается объект.

Объектный подход к программированию

- Применяя инкапсуляцию, мы защищаем данные, принадлежащие объекту, от возможных ошибок, которые могут возникнуть при прямом доступе к этим данным. Кроме того, применение этого принципа очень часто помогает локализовать возможные ошибки в коде программы. А это намного упрощает процесс поиска и исправления этих ошибок. Можно сказать, что инкапсуляция подразумевает под собой скрытие данных, что позволяет защитить эти данные. Однако применение инкапсуляции ведет к снижению эффективности доступа к элементам объекта. Это обусловлено необходимостью вызова методов для изменения внутренних элементов (переменных) объекта. Но при современном уровне развития вычислительной техники эти потери в эффективности не играют существенной роли.

Объектный подход к программированию

- *Наследование* – это процесс, посредством которого один объект может наследовать свойства другого объекта и добавлять к ним черты, характерные только для него. В итоге создаётся иерархия объектных типов, где поля данных и методов «предков» автоматически являются и полями данных и методов «потомков».
- Смысл и универсальность наследования заключается в том, что не надо каждый раз заново («с нуля») описывать новый объект, а можно указать «родителя» (базовый класс) и описать отличительные особенности нового класса. В результате новый объект будет обладать всеми свойствами родительского класса плюс своими собственными отличительными особенностями.

Объектный подход к программированию

- *Полиморфизм* – это свойство, которое позволяет одно и то же имя использовать для решения нескольких технически разных задач. Полиморфизм подразумевает такое определение методов в иерархии типов, при котором метод с одним именем может применяться к различным родственным объектам. В общем смысле концепцией полиморфизма служит идея «один интерфейс – множество методов». Преимуществом полиморфизма является то, что он помогает снижать сложность программ, разрешая использование одного интерфейса для единого класса действий. Выбор конкретного действия в зависимости от ситуации возлагается на компилятор.

Четвертый этап – компонентный подход

- Компонентный подход предполагает построение программного обеспечения из отдельных компонентов физически отдельно существующих частей программного обеспечения, которые взаимодействуют между собой через стандартизованные двоичные интерфейсы. В отличие от обычных объектов объекты-компоненты можно собрать в динамически вызываемые библиотеки или исполняемые файлы, распространять в двоичном виде (без исходных текстов) и использовать в любом языке программирования, поддерживающем соответствующую технологию. Это позволяет программистам создавать продукты, хотя бы частично состоящие из повторно использованных частей, т.е. применять технологию, хорошо зарекомендовавшую себя в области проектирования аппаратуры.

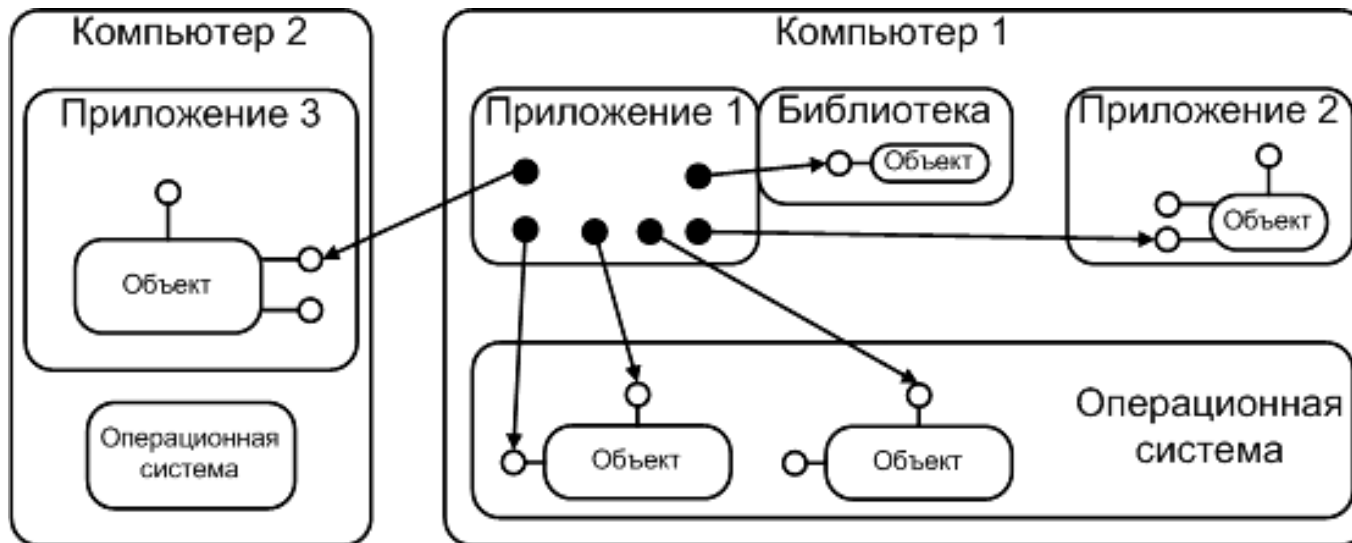
Компонентный подход

- Компонентный подход лежит в основе технологий, разработанных на базе COM (Component Object Model – компонентная модель объектов), и технологии создания распределённых приложений CORBA (Common Object Request Broker Architecture – общая архитектура с посредником обработки запросов объектов). Эти технологии используют сходные принципы и различаются лишь особенностями их реализации.

Компонентный подход

- Технология COM фирмы Microsoft является развитием технологии OLE I (Object Linking and Embedding – связывание и внедрение объектов), которая использовалась в ранних версиях Windows для создания составных документов. Технология COM определяет общую парадигму взаимодействия программ любых типов: библиотек, приложений, операционной системы, т.е. позволяет одной части программного обеспечения использовать функции (службы), предоставляемые другой, независимо от того, функционируют ли эти части в пределах одного процесса, в разных процессах на одном компьютере или на разных компьютерах. Модификация COM, обеспечивающая передачу вызовов между компьютерами, называется DCOM (Distributed COM – распределённая COM).

Взаимодействие программных компонентов различных типов



Компонентный подход

- По технологии COM приложение предоставляет свои службы, используя специальные объекты – *объекты COM*, которые являются экземплярами *классов COM*. Объект COM так же, как обычный объект, включает поля и методы, но в отличие от обычных объектов каждый объект COM может реализовывать несколько интерфейсов, обеспечивающих доступ к его полям и функциям. Это достигается за счет организации отдельной таблицы адресов методов для каждого интерфейса (по типу таблиц виртуальных методов). При этом интерфейс обычно объединяет несколько однотипных функций. Кроме того, классы COM поддерживают *наследование интерфейсов*, но не поддерживают *наследования реализации*, т.е. не наследуют код методов, хотя при необходимости объект класса-потомка может вызвать метод родителя.

Компонентный подход

- Каждый интерфейс имеет имя, начинающееся с символа «I», и глобальный уникальный идентификатор IID (Interface Identifier). Любой объект COM обязательно реализует интерфейс IUnknown (на схемах этот интерфейс всегда располагают сверху). Использование этого интерфейса позволяет получить доступ к остальным интерфейсам объекта.
- Объект всегда функционирует в составе *сервера* – динамической библиотеки или исполняемого файла, которые обеспечивают функционирование объекта.

Различают три типа серверов:

- внутренний сервер – реализуется динамическими библиотеками, которые подключаются к приложению-клиенту и работают в одном с ними адресном пространстве, – наиболее эффективный сервер, кроме того, он не требует специальных средств;
- локальный сервер – создается отдельным процессом (модулем, exe), который работает на одном компьютере с клиентом;
- удаленный сервер – создается процессом, который работает на другом компьютере.

Например, Microsoft Word является локальным сервером. Он включает множество объектов, которые могут использоваться другими приложениями.

Компонентный подход

- Для обращения к службам клиент должен получить указатель на соответствующий интерфейс. Перед первым обращением к объекту клиент посылает запрос к библиотеке COM, хранящей информацию обо всех зарегистрированных в системе классах COM объектов, и передает ей имя класса, идентификатор интерфейса и тип сервера. Библиотека запускает необходимый сервер, создает требуемые объекты и возвращает указатели на объекты и интерфейсы. Получив указатели, клиент может вызывать необходимые функции объекта.

Компонентный подход

- Взаимодействие клиента и сервера обеспечивается базовыми механизмами COM или DCOM, поэтому клиенту безразлично местонахождение объекта. При использовании локальных и удаленных серверов в адресном пространстве клиента создается *проху-объект* – заместитель объекта COM, а в адресном пространстве сервера COM – заглушка, соответствующая клиенту. Получив задание от клиента, заместитель упаковывает его параметры и, используя службы операционной системы, передает вызов заглушке. Заглушка распаковывает задание и передает его объекту COM. Результат возвращается клиенту в обратном порядке.

Компонентный подход

- На базе технологии COM и её распределённой версии DCOM были разработаны компонентные технологии, решающие различные задачи разработки программного обеспечения.
- OLE-automation или просто Automation (автоматизация), – технология создания программируемых приложений, обеспечивающая программируемый доступ к внутренним службам этих приложений.
- ActiveX – технология, построенная на базе OLE-automation, предназначена для создания программного обеспечения как сосредоточенного на одном компьютере, так и распределённого в сети. Предполагает использование визуального программирования для создания компонентов – элементов управления ActiveX.

Компонентный подход

- Полученные таким образом элементы управления можно устанавливать на компьютер дистанционно с удалённого сервера, причём устанавливаемый код зависит от используемой операционной системы. Это позволяет применять элементы управления ActiveX в клиентских частях приложений Интернет. Технология CORBA, разработанная группой компаний OMC (Object Management Group – группа внедрения объектной технологии программирования), реализует подход, аналогичный COM, на базе объектов и интерфейсов CORBA. Программное ядро CORBA реализовано для всех основных аппаратных и программных платформ, и потому эту технологию можно использовать для создания распределённого программного обеспечения в гетерогенной (разнородной) вычислительной среде. Организация взаимодействия между объектами клиента и сервера в CORBA осуществляется с помощью специального посредника, названного VisiBroker, и другого специализированного программного обеспечения.

Выводы

- Отличительной особенностью современного этапа развития технологии программирования, кроме изменения подхода, является создание и внедрение автоматизированных технологий разработки и сопровождения программного обеспечения, которые были названы CASE-технологиями (Computer-Aided Software/System Engineering – разработка программного обеспечения/программных систем с использованием компьютерной поддержки). Без средств автоматизации разработка достаточно сложного программного обеспечения на настоящий момент становится трудно осуществимой. На сегодня существуют CASE-технологии, поддерживающие как структурный, так и объектный (в том числе и компонентный) подходы к программированию.

Спасибо за внимание!

Ищенко Алексей Петрович